

Matt Pocock Skills — AI-Assisted Engineering Playbook

Executive take

Matt Pocock's skills are most useful as an **engineering control system for agents**, not as a list of slash commands. Their job is to make an AI coding loop behave like a good senior engineer: establish the real problem, expose decisions, create a testable contract, make a small verified change, then review against both the contract and repository standards.

The default spine is:

```
grill-with-docs → to-spec → to-tickets → implement + tdd → code-review
```

Use the whole spine only when uncertainty, blast radius, or duration justifies it. A small known fix should not be buried in planning theater. A multi-service platform should not be "one big prompt" either.

The operating model

Work characteristic	Correct agent posture	Skills that earn their keep
Small and understood	Make one narrow change and prove it	implement , tdd , code-review
Requirements or vocabulary unclear	Resolve decisions before code	grill-with-docs , domain-modeling , research
Risky or cross-cutting	Produce a written behavioral contract	to-spec , codebase-design
Too large for one context	Slice into independently demonstrable work	to-tickets , handoff
A key assumption is unproven	Run a throwaway experiment	prototype , then to-spec
Existing behavior is wrong	Reproduce first; fix second	diagnosing-bugs , tdd
System is getting harder to change	Improve a high-leverage seam, not everything	improve-codebase-architecture , codebase-design

Complete skill catalog — what each skill is and when to use it

This is the full set from the studied snapshot: **25 promoted daily-use skills**, **4 niche utilities**, and **9 in-progress skills**. Treat the promoted set as the working toolbox. Treat misc and in-progress skills as deliberate opt-ins, not automatic standards.

Promoted engineering skills

Skill	What it is	When to use it	Typical role in an AI-assisted flow
setup-matt-pocock-skills	Repository setup wizard for tracker, labels, docs locations, and agent instructions.	Once per repository or when workflow conventions change.	Establishes the operating contract for downstream skills.
ask-matt	Skill router.	You know the situation but not the right workflow.	Picks a sensible entry point; it does not implement work.
grill-with-docs	Repository-aware decision interview plus glossary/ADR upkeep.	Starting meaningful work in an existing codebase.	Align on intent, invariants, vocabulary, and non-goals before design.
to-spec	Converts settled decisions into a behavioral, testable specification.	Agreement exists but the work is not yet buildable.	Defines scope, acceptance behavior, test seams, decisions, and exclusions.
to-tickets	Breaks a spec into small, dependency-aware tracer-bullet tickets.	Work exceeds one context window or needs safe parallelism.	Makes the unblocked implementation frontier visible.
implement	Delivery wrapper that builds, checks, reviews, and commits.	A bounded ticket or spec is ready.	Executes work with tests and final verification rather than stopping at code generation.
tdd	Red-green-refactor discipline centered on public behavior seams.	Building behavior or fixing a reproducible regression.	Writes a failing behavior test, makes the smallest fix, then repeats.
code-review	Independent standards and spec review from a fixed diff point.	Before merge, on a PR, or after an implementation slice.	Checks both “is this good code?” and “did it build the requested thing?”
diagnosing-bugs	Evidence-first debugging loop.	A bug, regression, failure, or slowdown is not understood.	Reproduce, minimize, test ranked hypotheses, add regression coverage, then fix.
improve-codebase-architecture	Survey for high-leverage design/deep-module opportunities.	Code is becoming harder to change or after a development surge.	Selects one worthwhile architectural improvement instead of a vague cleanup rewrite.
codebase-design	Shared vocabulary for deep modules, interfaces, seams,	Designing boundaries, refactoring modules, or	Keeps public interfaces small and implementation complexity

Skill	What it is	When to use it	Typical role in an AI-assisted flow
	locality, and adapters.	choosing test seams.	hidden.
domain-modeling	Ubiquitous-language and ADR discipline.	Terms are overloaded, domain boundaries are fuzzy, or a hard trade-off needs recording.	Improves naming, navigation, and decision memory.
research	Primary-source research that produces cited repository notes.	APIs, vendors, frameworks, or facts need validation before design.	Supplies evidence; it does not replace architecture decisions.
prototype	Deliberately throwaway runnable experiment.	A key state model, integration, performance, or UI assumption is uncertain.	Produces evidence before production design; do not merge it as product code.
wayfinder	Decision-map workflow for large, foggy initiatives.	A system or program cannot be honestly planned in one session.	Separates unresolved decision tickets from implementation-ready work.
triage	Issue/PR intake and state-machine management.	An incoming queue needs evidence, ownership, labels, or actionable briefs.	Verifies claims, identifies duplicates, and prepares work for agents.
resolving-merge-conflicts	Intent-preserving merge/rebase conflict resolution.	A Git conflict already exists.	Traces both sides to their source intent, resolves, tests, and completes the operation.
wizard	Generates an interactive Bash guide for human-only steps.	A person must enter secrets, use a dashboard, provision, migrate, or cut over.	Creates a safe, confirmable human execution path rather than pretending the agent can do it.

Promoted productivity skills

Skill	What it is	When to use it	Typical role
grill-me	Stateless user-facing decision interview.	A non-repository plan or design is still fuzzy.	Use instead of premature planning; prefer grill-with-docs for codebase work.
grilling	The reusable interview primitive behind several other skills.	The agent needs to resolve a decision tree without guessing.	Model-invoked foundation for structured questions and explicit choices.
handoff	Compact continuation record for another session or agent.	You need to pause or transfer partially complete work.	Preserves evidence, risks, artifacts, and next action without bloating prompts.
teach	Stateful, multi-session learning workflow.	You want durable technical learning instead of an isolated explanation.	Builds lessons, retrieval practice, resources, and learning records.

Skill	What it is	When to use it	Typical role
to-questionnaire	Markdown question set for a decision owner.	Progress is blocked by information someone else holds.	Turns “ask them” into an async or meeting-ready instrument.
wait-what	Plain-English re-explanation using project vocabulary.	An agent explanation did not land.	Immediate clarity recovery; not a design or implementation step.
writing-for-agents	Guidance for skills, AGENTS.md, CLAUDE.md, and pointed-to docs.	Agent-facing instructions are stale, bloated, or inconsistent.	Applies progressive disclosure and explicit completion bounds.

Misc skills — useful, but niche

Skill	What it is	When to use it	Boundary
git-guardrails-claude-code	Claude Code hook that blocks risky Git actions.	You want preventative protection against push/reset/clean/discard mistakes.	Claude-specific; merge with existing hooks and test the rule set.
migrate-to-shoehorn	Converts test fixtures/assertions to Total TypeScript ShoeHorn helpers.	TypeScript tests need concise but type-aware partial fixtures.	Test code only; do not use it as production validation.
scaffold-exercises	Creates AI Hero course-exercise structure and validates it.	Authoring AI Hero exercises.	Product-specific, not a general application scaffolder.
setup-pre-commit	Adds Husky, lint-staged, Prettier, and detected checks.	A JavaScript/TypeScript repo lacks commit-time hygiene.	Inspect existing package-manager and CI conventions before merging it.

In-progress skills — public beta, not foundational process

Skill	What it is	When to use it	Caution
claude-handoff	Starts a new background Claude agent with a concise continuation prompt.	A Claude-specific continuation should begin immediately.	Claude-specific; redact secrets in the prompt.
implement-spec	Parallel worktree implementation of a complete spec/ticket graph.	The spec and dependency graph are already stable and ready.	Operationally complex; do not use it to resolve uncertainty.
loop-me	Multi-session workflow-spec design tool.	Designing a repeatable work/life loop.	Produces a spec, not necessarily automation.
pr	Concise PR body with summary, evidence, and merge danger.	Preparing a reviewable pull request.	A PR description is not a substitute for tests or review.
retro	Proposed agent-environment retrospective workflow.	Reviewing recurring friction after coding sessions.	Explicitly a stub/design note in this snapshot.

Skill	What it is	When to use it	Caution
setup-ts-deep-modules	Dependency-cruiser setup to enforce TypeScript package entry points.	A TypeScript repo needs enforceable deep-module boundaries.	Merge configuration carefully; do not make unrelated tsconfig changes.
writing-fragments	Append-only raw-idea writing workspace.	Gathering material before article structure exists.	Not an engineering design/planning workflow.
writing-beats	Shapes fixed raw material into coherent article beats.	Exploration is complete but narrative order is unclear.	Writing workflow only.
writing-shape	Turns raw Markdown into a separately structured article.	Source material is fixed and a deliberate article is needed.	Read-only input discipline; not a publishing system.

The five skills to internalize first

1. `grill-with-docs` — decide what to build and learn the local language.
2. `to-spec` — turn the decision into a testable contract.
3. `tdd` — make behavior observable before implementation grows.
4. `implement` — deliver with focused checks and a full verification pass.
5. `code-review` — independently verify spec fidelity and engineering standards.

Everything else is a targeted amplifier: use it because the problem demands it, not because the catalog exists.

Common engineering use cases

1. Add a new feature to an existing codebase

Typical risk: an agent implements the UI/API requested but violates an existing domain rule, architectural boundary, or operational convention.

Recommended route

```
grill-with-docs → to-spec → [to-tickets if >1 session] → implement + tdd → code-review
```

How the skills help

- `grill-with-docs` forces the agent to read the existing repository and establish terms, invariants, ownership, and non-goals before proposing architecture.
- `to-spec` turns the conversation into observable user stories, acceptance behavior, boundaries, decisions, and test seams.
- `to-tickets` creates vertical slices when a feature spans backend, API, UI, migrations, or operational work.
- `tdd` makes each slice demonstrate behavior through a public seam rather than internal implementation details.
- `code-review` runs independent **spec** and **standards** reviews, so a clean diff cannot hide a missed requirement.

Example

Add a team-invite workflow: first define invitation states, expiry, idempotency, authorization, notification failure behavior, and audit expectations. Then slice as: persistence + domain service; API contract; email delivery adapter; UI; observability/rollout. Do not start by asking an agent to "build invites end to end."

Principal-engineer guardrail: write the contract before the schema migration. Data shape is implementation; behavior and rollback are the design.

2. Refactor an existing feature

Typical risk: a cosmetically cleaner rewrite changes hidden behavior, removes useful compatibility, or leaves test seams worse than before.

Recommended route

```
codebase-design → grill-with-docs → tdd characterization tests → implement → code-review
```

For a broad mechanical migration, use an **expand–contract** plan rather than fake vertical tickets.

How the skills help

- `codebase-design` applies deep-module reasoning: reduce the public interface while hiding complexity behind it.
- `grill-with-docs` identifies what behavior must remain stable and whether the domain vocabulary is misleading.
- `tdd` creates characterization tests at public seams before structural work begins.
- `improve-codebase-architecture` helps select the one high-leverage hotspot instead of launching a vague cleanup campaign.

Example

Extract a payment-provider adapter from a sprawling checkout service. First characterize charge, retry, refund, idempotency, error mapping, and telemetry behavior. Introduce an adapter behind the existing interface, migrate callers, measure, then remove the old path. Do not move files until the behavioral boundary is explicit.

Principal-engineer guardrail: preserve behavior first. Refactoring is not permission to redesign product semantics unnoticed.

3. Fix a bug, regression, or performance problem

Typical risk: agents invent causes, apply plausible patches, and claim success without a reproducible loop.

Recommended route

```
diagnosing-bugs → tdd regression test → implement → code-review
```

How the skills help

- `diagnosing-bugs` requires reproduction/minimization before hypotheses, then tests ranked falsifiable explanations one at a time.
- `tdd` locks in the regression at the public behavior boundary before the fix.

- `code-review` checks whether the patch fixes the original failure rather than merely changing nearby code.

Example

An intermittent duplicate webhook: capture the exact delivery shape and concurrency timing, identify idempotency storage behavior, write a duplicate-delivery regression test, then modify the narrow persistence/sequencing boundary.

Principal-engineer guardrail: for performance, measure and bisect. Logging until something feels true is not diagnosis.

4. Infrastructure as Code — Terraform, Pulumi, CDK, Kubernetes

Typical risk: a generated plan looks syntactically valid but creates an outage, destructive replacement, secret leak, public exposure, or region/policy violation.

Recommended route

```
research → grill-with-docs → to-spec → to-tickets → implement → code-review → human plan/apply gate
```

How the skills help

- `research` grounds provider/service assumptions in primary documentation and existing stack conventions.
- `grill-with-docs` makes environment, account, region, tenancy, networking, ownership, and rollback decisions explicit.
- `to-spec` records desired state, security invariants, validation plan, migration sequence, and rollback conditions.
- `to-tickets` separates prerequisites such as IAM, network, data migration, service deployment, and observability.
- `code-review` should include a dedicated **blast-radius and destroy/replace** pass in addition to normal standards/spec review.

Example

Provision a private AWS service: specify the Jakarta-region constraint, VPC/subnet placement, IAM least privilege, encryption, secret references, loopback/private ingress, health checks, backups, observability, and a `terraform plan` review gate. Apply is a human-confirmed operation, not an agent default.

Principal-engineer guardrail: Terraform apply is a two-way-door action only when a tested rollback exists. Treat destructive replacements, databases, IAM, DNS, and public ingress as explicit approval gates.

5. Build scratch code from research results

Typical risk: research findings become production-shaped code before the core assumption has been validated.

Recommended route

```
research → prototype → grill-with-docs → to-spec → implement
```

How the skills help

- `research` gathers primary-source facts and writes cited notes.
- `prototype` asks one precise question, builds only enough runnable code to answer it, and records the verdict.
- `grill-with-docs` separates what the experiment proved from remaining design decisions.
- `to-spec` converts the surviving decision into a production contract.

Example

Evaluate an AI document-extraction pipeline: prototype extraction quality, latency, cost, privacy boundary, and failure handling against representative documents. Production work starts only once the chosen model, fallback, evaluation threshold, and data-retention decision are explicit.

Principal-engineer guardrail: prototypes are disposable evidence. Do not merge the prototype as the first production version because it "already works."

6. Build a system from scratch with multiple services

Typical risk: one agent prompt produces a diagram and a pile of components with unclear ownership, deployment sequencing, interfaces, security boundaries, and operational responsibility.

Recommended route

```
wayfinder → research → grill-with-docs + domain-modeling → prototype where needed → to-spec → to-tickets → implement → code-review
```

How the skills help

- `wayfinder` maps uncertainty as decision tickets and keeps fog separate from ready work.
- `domain-modeling` establishes bounded contexts, ubiquitous language, and the few decisions worth ADRs.
- `prototype` validates sharp architectural uncertainties: state model, integration shape, UI direction, throughput, or deployment topology.
- `to-spec` defines system behavior, service responsibilities, contracts, non-functional requirements, failure modes, and rollout criteria.
- `to-tickets` makes dependencies visible and identifies the safe parallel frontier.
- `handoff` preserves execution state between sessions and agents without stuffing the full history into prompts.

Example

Build an event-driven SaaS platform: begin with actors, tenancy, authorization, data ownership, sync vs async boundaries, consistency expectations, operational SLOs, and incident ownership. Create a thin vertical walking skeleton from authenticated request through one durable action, event, read model, metric, and deployment. Expand only after that path is proved.

Principal-engineer guardrail: do not start with a microservice count. Start with boundaries, ownership, and a thin end-to-end slice. Split services only where the operational and domain boundary earns the cost.

7. Integrate a third-party API or replace a vendor

Typical risk: the happy path works while retries, quotas, pagination, version changes, auth refresh, webhook duplication, and vendor outages are ignored.

Recommended route

```
research → grill-with-docs → to-spec → prototype or contract tests → implement + tdd → code-review
```

Key decisions to make explicit

- Who owns retries, backoff, idempotency, and dead-letter handling?
- What are the timeout, quota, and cost boundaries?
- Which vendor data is source of truth?
- How are credentials stored and rotated?
- What is the fallback or exit strategy?
- Which behavior is contract-tested against the real vendor or a verified sandbox?

Principal-engineer guardrail: isolate a vendor behind a small adapter only when it buys real replacement, testability, or policy control. Do not create a ceremonial abstraction with one implementation and no meaningful seam.

8. Dependency upgrade, framework migration, or security remediation

Typical risk: an apparent version bump changes runtime behavior, generated artifacts, lockfiles, plugin compatibility, deployment requirements, or data migrations.

Recommended route

```
research → diagnosing-bugs / grill-with-docs → to-spec → to-tickets → implement → code-review
```

How to use the skills

- Research changelogs, migration guides, source compatibility notes, and breaking-change ownership.
- Write a migration spec with compatibility target, rollback boundary, smoke tests, contract tests, and release sequence.
- Ticket expand—contract stages separately: dual read/write, backfill, cutover, observation, and removal.
- Review against both migration plan and production risk.

Principal-engineer guardrail: a green unit suite does not prove an upgrade. Verify actual version, startup, auth, integrations, migrations, and deployment behavior.

9. Code review, PR preparation, and release readiness

Typical risk: one reviewer checks style while nobody verifies the original behavior, operational impact, or rollback.

Recommended route

```
code-review → pr → release/change gate
```

Use review on two independent axes

1. **Spec review:** Are acceptance criteria, negative cases, non-goals, and migration/operational requirements satisfied?
2. **Standards review:** Does the code fit repository conventions, security, architecture, tests, and maintainability expectations?

Add a third release axis for consequential changes: deploy/rollback path, data safety, observability, runbook, and blast radius.

Principal-engineer guardrail: do not write a PR description as marketing. State evidence, remaining risk, and whether the change is one-way or two-way.

10. Long-running work across agents, sessions, or parallel streams

Typical risk: duplicated research, conflicting edits, vague ownership, and context loss.

Recommended route

```
to-tickets / wayfinder → handoff → implement-spec (beta, only when ready) → code-review
```

How the skills help

- `to-tickets` creates independently testable, dependency-aware slices.
- `wayfinder` keeps unresolved decisions out of the implementation queue.
- `handoff` records only active context, current evidence, artifacts, risks, and next action.
- `implement-spec` is an experimental parallel implementation harness; use it only after a stable spec/ticket graph exists.

Principal-engineer guardrail: parallelism accelerates ready work, not uncertainty. Never delegate a vague architecture decision to multiple agents and hope the merge is the answer.

Recommended recipes by scope

Small change — under one session

```
Read relevant code → grill briefly → implement + TDD → focused tests → code review → commit
```

Use when scope and acceptance behavior are already clear.

Medium feature — one team / one repository

```
grill-with-docs → to-spec → implement in vertical slices → code-review → release check
```

Use when the feature crosses at least two layers or changes durable behavior.

Large initiative — multi-service or multi-session

wayfinder → research + prototypes → spec → ticket graph → guarded parallel implementation → independent review → staged rollout

Use when decisions and work have different readiness levels.

Operationally sensitive change

research → spec with invariants and rollback → dry run/plan → human approval → narrow apply → health verification → evidence record

Use for IaC, production databases, credentials, access control, DNS, queues, and irreversible migrations.

What to put in the repository

The skills work best when the repository gives agents a small durable truth layer:

- a concise `AGENTS.md` or existing agent instruction file that points to deeper docs;
- a glossary/context map for domain terms;
- ADRs only for hard-to-reverse, surprising decisions;
- a clear test/run/build command inventory;
- a definition of done including tests, review, and operational evidence;
- a tracker or local ticket convention for work that exceeds one context window.

Do not dump every past decision into a root instruction file. Use progressive disclosure: short entry point, precise pointers, current sources of truth.

Principal-engineer rules for AI-assisted work

1. **Make decisions visible before implementation.** Agents are fast at filling gaps; that is exactly why hidden gaps are dangerous.
2. **Specify behavior and invariants, not a preferred file layout.** Let implementation follow the contract.
3. **Choose seams before writing tests.** Tests should protect public behavior and survive refactors.
4. **Prefer a thin end-to-end slice over horizontal layers.** A running narrow path exposes integration risk early.
5. **Keep a rollback boundary for stateful or external changes.** Backups, plans, migration phases, and health checks are engineering work.
6. **Use evidence as the completion condition.** A command exited zero is not proof that the system works.
7. **Treat security and operations as requirements.** Authentication, data handling, observability, capacity, recovery, and ownership belong in the spec.
8. **Use the shortest process that protects the decision.** Process is a risk-control tool, not a performance.

Practical prompts for an AI coding agent

New feature

Use `grill-with-docs` first. Read the existing code and docs, identify unresolved product and technical decisions, and propose a small behavioral spec with test seams before editing code. Keep the first implementation slice vertical and demonstrable.

Refactor

Use `codebase-design` and create characterization tests at public seams. Preserve behavior first. Propose an expand–contract sequence and show what interface becomes smaller or clearer; do not begin with file moves.

IaC

Research official provider documentation and existing infrastructure conventions. Produce a spec covering region, tenancy, IAM, ingress/egress, secrets, state, plan/apply gates, observability, data migration, and rollback. Do not apply changes without an explicit human gate.

System from scratch

Start with `wayfinder`: separate ready decisions from fog. Define actors, boundaries, data ownership, failure behavior, and SLOs. Build a thin vertical walking skeleton before adding services or broad abstractions.

Adoption plan

Start with five skills: `grill-with-docs`, `to-spec`, `tdd`, `implement`, and `code-review`.

After one meaningful feature, add `to-tickets`, `diagnosing-bugs`, and `handoff`. Add `research`, `domain-modeling`, `prototype`, and `improve-codebase-architecture` when recurring ambiguity or design friction proves they are needed.

Do not install or invoke every skill by default. The outcome we want is not ritual. It is code that is easier to change, easier to verify, and harder to accidentally break.

Sources

- Matt Pocock Skills repository: <https://github.com/mattpocock/skills>
- AI Hero skills documentation: <https://www.aihero.dev/skills>
- Original guide supplied by Irfan: *Matt Pocock's Skills: a Practical Learning Guide* (repository snapshot `c55ee46073ed923f86ce59a5eb3b6d895095d1b7`, 18 Sep 2026)